UNIT 3
Strings, Lists, Tuples, and Dictionaries

Prof. Sheekha Raina
Assistant Professor
AI & ML Department

Content

1. String operations and methods……21
2. Lists……23
3. Lists: indexing, slicing, methods……34
4. Tuples……38
5. Tuples: creation, operations, immutability……45
6. Sets and frozensets……57
7

Strings in Python:

A string is a sequence of characters that can be a combination of letters, numbers, and special characters. It can be declared in python by using single quotes, double quotes, or even triple quotes. These quotes are not a part of a string; they define only starting and ending of the string. Strings are immutable, i.e., they cannot be changed. Each element of the string can be accessed using indexing or slicing operations.

```
# Assigning string to a variable
a = 'This is a string'
print (a)
b = "This is a string"
print (b)
c= '''This is a string'''
print (c)
```

Output:

```
This is a string
This is a string
This is a string
```

Creating a String

Strings can be created using either **single ('...')** or **double ("...")** quotes. Both behave the same.

Example: Creating two equivalent strings one with single and other with double quotes.

```
s1 = 'GfG'  # single quote
s2 = "GfG"  # double quote
print(s1)
print(s2)
```

Output

GfG
GfG

Multi-line Strings

Use triple quotes **('''...''')** or **("""..."""**) for strings that span multiple lines. Newlines are preserved.

Example: Define and print multi-line strings using both styles.

```
s = """I am Learning
Python String on GeeksforGeeks"""
print(s)
s = '''I'm a
Geek'''
print(s)
```

Output

I am Learning
Python String on GeeksforGeeks
I'm a
Geek

Accessing characters in String

Strings are indexed sequences. Positive indices start at **0** from the **left**; negative indices start at **-1** from the **right** as represented in below image:

0	1	2	3	4	5	6	7	8	9	10	11	12
G	E	E	K	S	F	O	R	G	E	E	K	S
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Indices of string in reverse

Example 1: Access specific characters through positive indexing

```
s = "GeeksforGeeks"
print(s[0])   # first character
print(s[4])   # 5th character
```

Output

G

s

Example 2: Read characters from the end using negative indices.

```
s = "GeeksforGeeks"
print(s[-10])   # 3rd character
print(s[-5])    # 5th character from end
```

Output

k
G

String Slicing

Slicing is a way to extract a portion of a string by specifying the **start** and **end** indexes. The syntax for slicing is **string[start:end]**, where **start** starting index and **end** is stopping index (excluded).

Example: In this example we are slicing through range and reversing a string.

```
s = "GeeksforGeeks"
print(s[1:4])   # characters from index 1 to 3
print(s[:3])    # from start to index 2
print(s[3:])    # from index 3 to end
print(s[::-1])  # reverse string
```

Output

eek
Gee
ksforGeeks
skeeGrofskeeG

String Iteration

Strings are iterable; you can loop through characters one by one.

Example: Here, it print each character on its own line.

```
s = "Python"
for char in s:
    print(char)
```

Output

```
P
y
t
h
o
n
```

Explanation: for loop pulls characters in order and each iteration prints the next character.

String Immutability

Strings are **immutable,** which means that they cannot be changed after they are created. If we need to manipulate strings then we can use methods like **concatenation, slicing** or **formatting** to create new strings based on original.

Example: In this example we are changing first character by building a new string.

```
s = “Rohan"
s = "G" + s[1:]   # create new string
print(s)
```

Output

Rohan

Deleting a String

In Python, it is not possible to delete individual characters from a string since strings are immutable. However, we can delete an entire string variable using the del keyword.

Example: Here, we are using del keyword to delete a string.

```
s = "GfG"
del s
```

Note: *After deleting the string if we try to access s then it will result in a* ***NameError*** *because variable no longer exists.*

Updating a String

As strings are immutable, “updates” create new strings using slicing or methods such as replace().

Example: This code fix the first letter and replace a word.

```
s = "hello geeks"
s1 = "H" + s[1:]                # update first character
s2 = s.replace(“Rohan", “Mohan")  # replace word
print(s1)
print(s2)
```

Output

Hello Rohan
hello Mohan

Explanation:

s1: slice from index 1 onward and prepend "H".
s2: replace(“Rohan", “Mohan") returns a new string.

Common String Methods

Python provides various built-in methods to manipulate strings. Below are some of the most useful methods:

1. **len()**: The len() function returns the total number of characters in a string (including spaces and punctuation).

Example:

```
s = “India"
print(len(s))
```

Output

5

2. **upper() and lower()**: upper() method converts all characters to uppercase whereas, lower() method converts all characters to lowercase.

Example:

```
s = "Hello World"
print(s.upper())
print(s.lower())
```

Output

HELLO WORLD
hello world

3. **strip() and replace()**: strip() removes leading and trailing whitespace from the string and replace() replaces all occurrences of a specified substring with another.

Example:

```
s = "   Gfg   "
print(s.strip())
s = "Python is fun"
print(s.replace("fun", "awesome"))
```

Output

Gfg

Python is awesome

Concatenating and Repeating Strings

We can concatenate strings using + operator and repeat them using * operator.

1. Strings can be combined by using **+ operator**.

Example: Join two words with a space.

```
s1 = "Hello"
s2 = "World"
print(s1 + " " + s2)
```

Output

Hello World

2. We can repeat a string multiple times using *** operator**.

Example: Repeat a greeting three times.

```
s = "Hello "
print(s * 3)
```

Output

Hello Hello Hello

Formatting Strings

Python provides several ways to include variables inside strings.

1. Using f-strings

The simplest and most preferred way to format strings is by using f-strings**.**

Example: Embed variables directly using {} placeholders.

```
name = "Alice"
age = 22
print(f"Name: {name}, Age: {age}")
```

Output

Name: Alice, Age: 22

2. Using format()

Another way to format strings is by using format() method.

Example: Use placeholders {} and pass values positionally.

```
s = "My name is {} and I am {} years old.".format("Alice", 22)
print(s)
```

Output

My name is Alice and I am 22 years old.

String Membership Testing

in keyword checks if a particular substring is present in a string.

Example: Here, we are testing for the presence of substrings.

```
s = "GeeksforGeeks"
print("Geeks" in s)
print("GfG" in s)
```

Output

True

False

Python Lists

In Python, a list is a built-in data structure that can hold an ordered collection of items.
Unlike arrays in some languages, Python lists are very flexible:
Can contain duplicate items

1. **Mutable:** items can be modified, replaced, or removed
2. **Ordered:** maintains the order in which items are added
3. **Index-based:** items are accessed using their position (starting from 0)

Can store mixed data types (integers, strings, booleans, even other lists

Creating a List

Lists can be created in several ways, such as using square brackets, the list() constructor or by repeating elements. Let's look at each method one by one with example:

1. Using Square Brackets

We use square brackets [] to create a list directly.

```
a = [1, 2, 3, 4, 5] # List of integers
b = ['apple', 'banana', 'cherry'] # List of strings
c = [1, 'hello', 3.14, True] # Mixed data types
print(a)
print(b)
print(c)
```

Output

```
[1, 2, 3, 4, 5]
['apple', 'banana', 'cherry']
[1, 'hello', 3.14, True]
```

2. Using list() Constructor

We can also create a list by passing an iterable (like a tuple, string or another list) to the list() function.

```
a = list((1, 2, 3, 'apple', 4.5))
print(a)

b = list("GFG")
print(b)
```

Output

[1, 2, 3, 'apple', 4.5]
['G', 'F', 'G']

3. Creating List with Repeated Elements

We can use the multiplication operator * to create a list with repeated items.

```
a = [2] * 5
b = [0] * 7

print(a)
print(b)
```

Output

[2, 2, 2, 2, 2]
[0, 0, 0, 0, 0, 0, 0]

Accessing List Elements

Elements in a list are accessed using indexing. Python indexes start at 0, so a[0] gives the first element. Negative indexes allow access from the end (e.g., -1 gives the last element).

```
a = [10, 20, 30, 40, 50]
print(a[0])
print(a[-1])
print(a[1:4])   # elements from index 1 to 3
```

Output

10
50
[20, 30, 40]

Adding Elements into List

We can add elements to a list using the following methods:

1. **append()**: Adds an element at the end of the list.
2. **extend()**: Adds multiple elements to the end of the list.
3. **insert()**: Adds an element at a specific position.
4. **clear()**: removes all items.

```
a = []
a.append(10)
print("After append(10):", a)
a.insert(0, 5)
print("After insert(0, 5):", a)
a.extend([15, 20, 25])
print("After extend([15, 20, 25]):", a)
a.clear()
print("After clear():", a)
```

Output

After append(10): [10]
After insert(0, 5): [5, 10]
After extend([15, 20, 25]): [5, 10, 15, 20, 25]
After clear(): []

Updating Elements into List

Since lists are mutable, we can update elements by accessing them via their index.

```
a = [10, 20, 30, 40, 50]
a[1] = 25
print(a)
```

Output

[10, 25, 30, 40, 50]

Removing Elements from List

We can remove elements from a list using:

remove(): Removes the first occurrence of an element.

pop(): Removes the element at a specific index or the last element if no index is specified.

del statement: Deletes an element at a specified index.

```
a = [10, 20, 30, 40, 50]
a.remove(30)
print("After remove(30):", a)
popped_val = a.pop(1)
print("Popped element:", popped_val)
print("After pop(1):", a)
del a[0]
print("After del a[0]:", a)
```

Output

After remove(30): [10, 20, 40, 50]
Popped element: 20
After pop(1): [10, 40, 50]
After del a[0]: [40, 50]

Conditional Statements in Python:

Iterating Over Lists

We can iterate over lists using loops, which is useful for performing actions on each item.

```
a = ['apple', 'banana', 'cherry']
for item in a:
    print(item)
```

Output

apple
banana
cherry

Nested Lists

A nested list is a list within another list, which is useful for representing matrices or tables. We can access nested elements by chaining indexes.

```
matrix = [ [1, 2, 3],
           [4, 5, 6],
           [7, 8, 9] ]
print(matrix[1][2])
```

Output

6

List Comprehension

List comprehension is a concise way to create lists using a single line of code. It is useful for applying an operation or filter to items in an iterable, such as a list or range.

```
squares = [x**2 for x in range(1, 6)]
print(squares)
```

Output

[1, 4, 9, 16, 25]

Explanation:

for x in range(1, 6): loops through each number from **1 to 5** (excluding 6).

x2:** squares each number x.

[ ]: collects all the squared numbers into a new list.

Python Tuples

A tuple in Python is an immutable ordered collection of elements.
Tuples are similar to lists, but unlike lists, they cannot be changed after their creation (i.e., they are immutable).
Tuples can hold elements of different data types.
The main characteristics of tuples are being ordered, heterogeneous and immutable.

Creating a Tuple

A tuple is created by placing all the items inside parentheses (), separated by commas. A tuple can have any number of items and they can be of different data types.

```
tup = ()
print(tup)
# Using String
tup = ('Geeks', 'For')
print(tup)
# Using List
li = [1, 2, 4, 5, 6]
print(tuple(li))
# Using Built-in Function
tup = tuple('Geeks')
print(tup)
```

Output

```
()
('Geeks', 'For')
(1, 2, 4, 5, 6)
('G', 'e', 'e', 'k', 's')
```

Creating a Tuple with Mixed Datatypes.

Tuples can contain elements of various data types, including other tuples, lists, dictionaries and even functions.

```
tup = (5, 'Welcome', 7, 'Geeks')
print(tup)
# Creating a Tuple with nested tuples
tup1 = (0, 1, 2, 3)
tup2 = ('python', 'geek')
tup3 = (tup1, tup2)
print(tup3)
# Creating a Tuple with repetition
tup1 = ('Geeks',) * 3
print(tup1)
# Creating a Tuple with the use of loop
tup = ('Geeks')
n = 5
for i in range(int(n)):
    tup = (tup,)
    print(tup)
```

Output

```
(5, 'Welcome', 7, 'Geeks')
((0, 1, 2, 3), ('python', 'geek'))
('Geeks', 'Geeks', 'Geeks')
('Geeks',)
(('Geeks',),)
((('Geeks',),),)
((((('Geeks',),),),),)
(((((('Geeks',),),),),),)
```

Python Tuple Basic Operations

Below are the Python tuple operations.

1. Accessing of Python Tuples
2. Concatenation of Tuples
3. Slicing of Tuple
4. Deleting a Tuple

Accessing of Tuples

We can access the elements of a tuple by using indexing and slicing, similar to how we access elements in a list. Indexing starts at 0 for the first element and goes up to n-1, where n is the number of elements in the tuple. Negative indexing starts from -1 for the last element and goes backward.

```
# Accessing Tuple with Indexing
tup = tuple("Geeks")
print(tup[0])
# Accessing a range of elements using slicing
print(tup[1:4])
print(tup[:3])
# Tuple unpacking
tup = ("Geeks", "For", "Geeks")
# This line unpack values of Tuple1
a, b, c = tup
print(a)
print(b)
print(c)
```

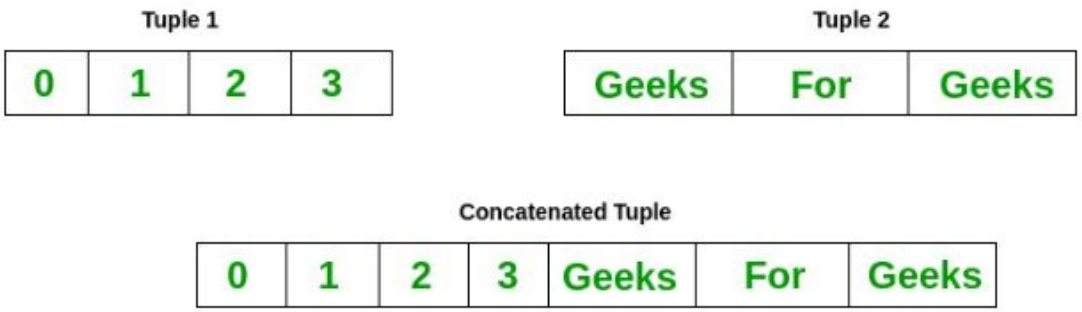
Output

G
('e', 'e', 'k')
('G', 'e', 'e')
Geeks
For
Geeks

Concatenation of Tuples

Tuples can be concatenated using the + operator. This operation combines two or more tuples to create a new tuple.

Note: *Only the same datatypes can be combined with concatenation, an error arises if a list and a tuple are combined*




```
tup1 = (0, 1, 2, 3)
tup2 = ('Geeks', 'For', 'Geeks')

tup3 = tup1 + tup2
print(tup3)
```

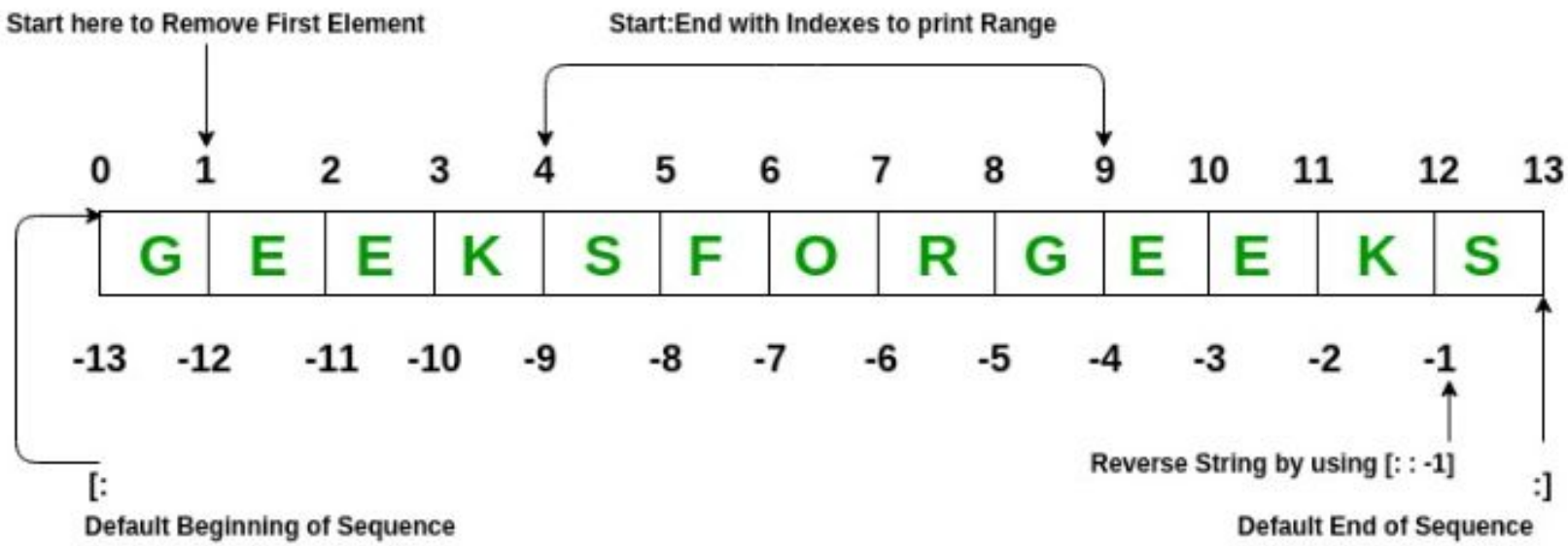
Output

```
(0, 1, 2, 3, 'Geeks', 'For',
'Geeks')
```

Slicing of Tuple

Slicing a tuple means creating a new tuple from a subset of elements of the original tuple. The slicing syntax is tuple[start:stop:step].

Note- *Negative Increment values can also be used to reverse the sequence of Tuples.*



```
tup = tuple('GEEKSFORGEEKS')
# Removing First element
print(tup[1:])
# Reversing the Tuple
print(tup[::-1])
# Printing elements of a Range
print(tup[4:9])
```

Output

```
('E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S')
('S', 'K', 'E', 'E', 'G', 'R', 'O', 'F', 'S', 'K', 'E', 'E', 'G')
('S', 'F', 'O', 'R', 'G')
```

Deleting a Tuple

Since tuples are immutable, we cannot delete individual elements of a tuple. However, we can delete an entire tuple using del statement.

Note: *Printing of Tuple after deletion results in an Error.*

```
tup = (0, 1, 2, 3, 4)
del tup

print(tup)
```

Output

```
ERROR!
Traceback (most recent call last):
File "<main.py>", line 6, in <module>
NameError: name 'tup' is not defined
```

Sets in Python

A Set in Python is used to store a collection of items with the following properties.

1. No duplicate elements. If try to insert the same item again, it overwrites previous one.
2. An unordered collection. When we access all items, they are accessed without any specific order and we cannot access items using indexes as we do in lists.
3. Internally use hashing that makes set efficient for search, insert and delete operations. It gives a major advantage over a list for problems with these operations.
4. Mutable, meaning we can add or remove elements after their creation, the individual elements within the set cannot be changed directly.

For Example

This example shows how to create a set with some numbers and check its type.

```
s = {10, 50, 20}
print(s)
print(type(s))
```

Output

{10, 50, 20}

<class 'set'>

***Note**: There is no specific order for set elements to be printed*

Type Casting

set() method in python is used to convert other data types, such as lists or tuples, into sets.

```
# typecasting list to set
s = set(["a", "b", "c"])
print(s)

# Adding element to the set
s.add("d")
print(s)
```

Output

```
{'c', 'b', 'a'}
{'d', 'c', 'b', 'a'}
```

Check unique and Immutable

Python sets cannot have duplicate values. While you cannot modify the individual elements directly, you can still add or remove elements from the set.

```
# a set cannot have duplicate values
s = {"Geeks", "for", "Geeks"}
print(s)

# values of a set cannot be changed
s[1] = "Hello"
print(s)
```

Output

{'Geeks', 'for'}

TypeError: 'set' object does not support item assignment

Explanation:

s = {"Geeks", "for", "Geeks"} duplicates are removed, only unique values remain.

s[1] = "Hello" Error, set elements cannot be changed directly. Only add() or remove() can modify a set.

Heterogeneous Element

Python sets can store heterogeneous elements in it, i.e., a set can store a mixture of string, integer, boolean, etc datatypes.

```
s = {"Geeks", "for", 10, 52.7, True}
print(s)
```

Output

```
{True, 52.7, 'Geeks', 10, 'for'}
```

Frozen Sets

A frozenset is an immutable version of a set. Its elements cannot be changed after creation, but you can perform operations like union, intersection and difference.
Use frozenset() to create one.

```
# Normal set (mutable)
s = set(["a", "b", "c"])
print("Normal Set:", s)
# Frozen set (immutable)
fs = frozenset(["e", "f", "g"])
print("Frozen Set:", fs)
```

Output

```
('Normal Set:', set(['a', 'c', 'b']))
('Frozen Set:', frozenset(['e', 'g', 'f']))
```

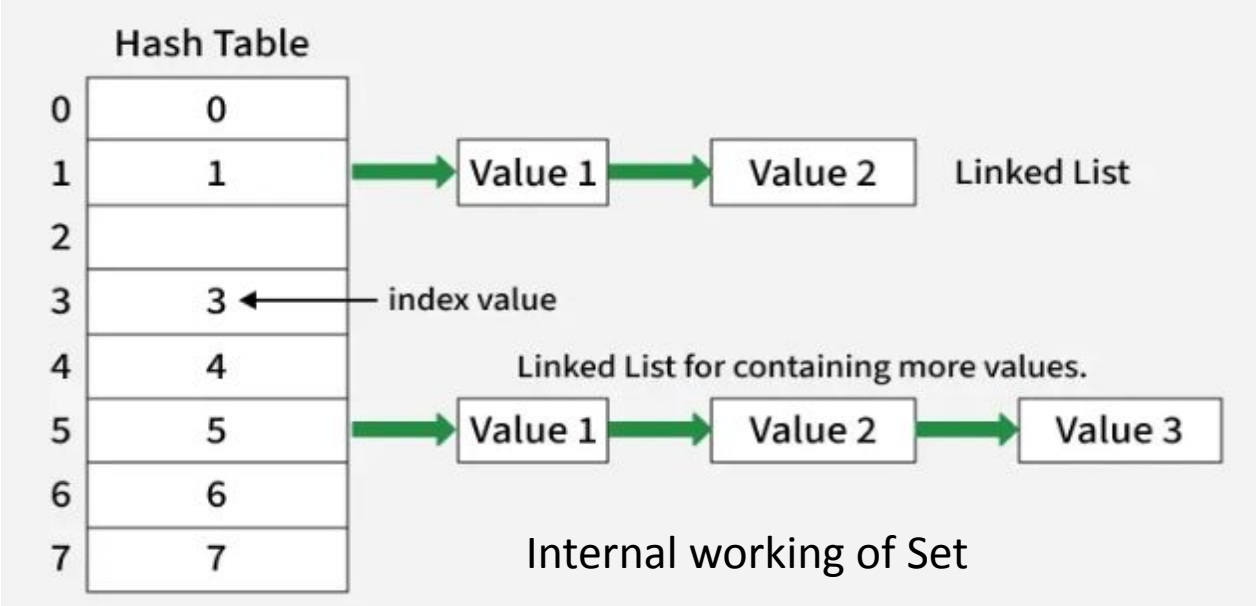
***Note**: Frozensets are immutable, so methods like add() or remove() cannot be used. They are also hashable, which allows them to be used as dictionary keys.*

Internal working of Set

This is based on a data structure known as a hash table. If Multiple values are present at the same index position, then the value is appended to that index position, to form a Linked List.

In, Python Sets are implemented using a dictionary with dummy variables, where key beings the members set with greater optimizations to the time complexity.

The diagram shows how Python internally manages collisions in sets using linked lists for efficient element storage and retrieval.



Internal working of Set

Conditional Statements in Python:

1. Each index in the hash table represents a possible hash value where elements are stored based on their computed hash key.
2. When two elements map to the same index, they are linked together using a linked list, forming a chain at that position.
3. This chaining mechanism helps efficiently handle collisions, allowing multiple elements to exist at the same hash index without overwriting each other.

Methods for Sets

1. Adding elements to Sets: add() function is used to insert new elements into a set. It automatically ignores duplicates.

```
s = {"a", "b", "c"}
s.add("d")
print(s)
```

Output

{'c', 'd', 'a', 'b'}

2. Union of Sets: union() function combines two sets and returns a new set with all unique elements.

```
a = {"x", "y"}
b = {"y", "z"}
u = a.union(b)
print(u)
```

Output

{'z', 'y', 'x'}

3. Intersection of Sets:

intersection() function returns a new set containing elements that are common to both sets.

```
a = {1, 2, 3}
b = {2, 3, 4}
i = a.intersection(b)
print(i)
```

Output

{2, 3}

4. Difference of Sets: difference() function returns a set containing elements that are in the first set but not in the second.

```
a = {1, 2, 3}
b = {2, 3, 4}
d = a.difference(b)
print(d)
```

Output

{1}

5. Clearing a Set:

clear() function removes all elements from a set, leaving it empty.

```
s = {1, 2, 3}
s.clear()
print(s)
```

Output

set()

frozenset() in Python

Python Method creates an immutable Set object from an iterable. It is a built-in Python function. As it is a set object, therefore, we cannot have duplicate values in the frozenset.
Example

In this example, we are creating a frozen set with the list in Python.

```
animals = frozenset(["cat", "dog", "lion"])
print("cat" in animals)
print("elephant" in animals)
```

Output
True
False

Python frozenset() Syntax

Syntax : *frozenset(iterable_object_name)*
Parameter : *iterable_object_name*
This function accepts iterable object as input parameter.
Return : *Returns an equivalent frozenset object.*

Python Dictionary

A Python dictionary is a data structure that stores data in key-value pairs, where each key is unique and is used to retrieve its associated value. It is mainly used when you want to store and access data by a name (key) instead of by position like in a list.

Example: This example shows how a dictionary stores data using keys and values.

```
data = { "name": "Jake", "age": 22 }
print(data)
```

Output

```
{'name': 'Jake', 'age': 22}
```

Explanation:

"name" and "age" are keys
"Jake" and 22 are their values
dictionary stores data in key : value format

Creating a Dictionary

A dictionary is created by writing key-value pairs inside { }, where each key is connected to a value using colon (:). A dictionary can also be created using the dict() function.

```
d1 = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(d1)
# using dict() constructor
d2 = dict(a = "Geeks", b = "for", c = "Geeks")
print(d2)
```

Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
{'a': 'Geeks', 'b': 'for', 'c': 'Geeks'}
```

Accessing Dictionary Items

A value in a dictionary is accessed by using its key. This can be done either with square brackets [ ] or with the get() method. Both return the value linked to the given key.

```
d = { "name": "Kat", 1: "Python", (1, 2): [1,2,4] }

# Access using key
print(d["name"])

# Access using get()
print(d.get("name"))
```

Output

```
Kat
Kat
```

Adding and Updating Dictionary Items

New items are added to a dictionary using the assignment operator (=) by giving a new key a value. If an existing key is used with the assignment operator, its value is updated with the new one.

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
# Adding a new key-value pair
d["age"] = 22
# Updating an existing value
d[1] = "Python dict"
print(d)
```

Output

{1: 'Python dict', 2: 'For', 3: 'Geeks', 'age': 22}

Removing Dictionary Items

Dictionary items can be removed using built-in deletion methods that work on keys:

1. **del**: removes an item using its key
2. **pop()**: removes the item with the given key and returns its value
3. **clear()**: removes and returns the last inserted key–value pair
4. **popitem()**: removes all items from the dictionary

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks', 'age':22}
```

```
# Using del
del d["age"]
print(d)
# Using pop()
val = d.pop(1)
print(val)
# Using popitem()
key, val = d.popitem()
print(f"Key: {key}, Value: {val}")
# Using clear()
d.clear()
print(d)
```

Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
Geeks
Key: 3, Value: Geeks
{}
```

Iterating Through a Dictionary

A dictionary can be traversed using a for loop to access its keys, values or both key-value pairs by using the built-in methods keys(), values() and items().

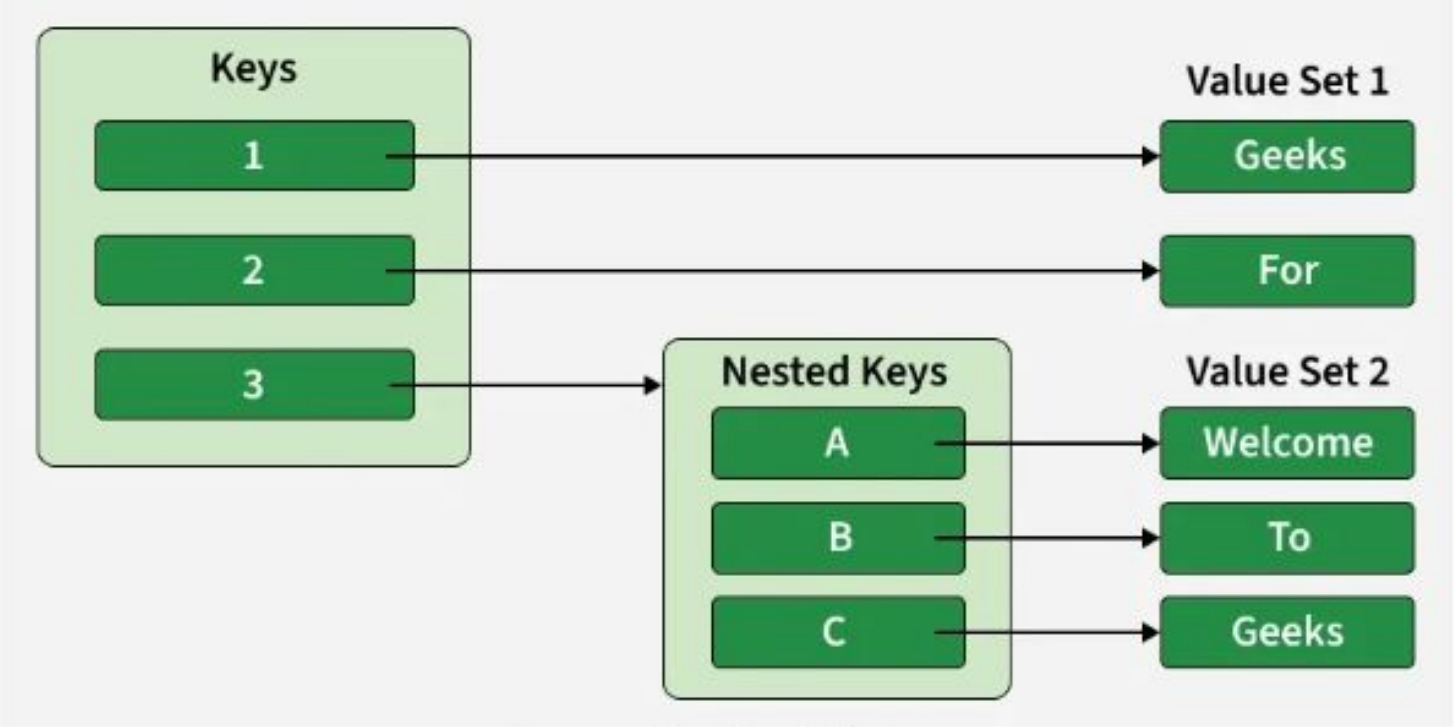
```
d = {1: 'Geeks', 2: 'For', 'age':22}
# Iterate over keys
for key in d:
    print(key)

# Iterate over values
for value in d.values():
    print(value)

# Iterate over key-value pairs
for key, value in d.items():
    print(f"{key}: {value}")
```

Output

```
1
2
age
Geeks
For
22
1: Geeks
2: For
age: 22
```

Nested Dictionaries

A nested dictionary is a dictionary that contains another dictionary as one of its values. Below diagram shows how a nested dictionary works, where key 3 points to another dictionary inside the main dictionary.



Representation of Nested Dictionary

Conditional Statements in Python:

The arrows show how each key is connected to its corresponding value.

```
d = {1: 'Geeks', 2: 'For', 3: {'A': 'Welcome', 'B': 'To', 'C': 'Geeks'}}
print(d)
```

Output

```
{1: 'Geeks', 2: 'For', 3: {'A': 'Welcome', 'B': 'To', 'C': 'Geeks'}}
```

Comprehensions in Python

Comprehensions in Python provide a concise and efficient way to create new sequences from existing ones. They enhance code readability and reduce the need for lengthy loops.

Why do we need Comprehensions

Encourages Modular Thinking: Promotes writing logic in smaller, reusable expressions.
Widely Used in Real-World Code: Found in data science, web development and automation scripts.
Easier Debugging & Testing: Fewer lines reduce the surface area for bugs.
Seamless with Other Python Features: Integrates well with functions like zip(), enumerate() and lambda.

Types of Comprehensions in Python

Python offers different types of comprehensions to simplify creation of data structures in a clean, readable way. Let’s explore each type with simple examples.

List Comprehensions

List comprehensions allow for the creation of lists in a single line, improving efficiency and readability. They follow a specific pattern to transform or filter data from an existing iterable.

Syntax:

[expression for item in iterable if condition]

Where:

a. **expression**: Operation applied to each item.
b. **item**: Variable representing the element from the iterable.
c. **iterable**: The source collection.
d. **condition (optional)**: A filter to include only specific items.

Example 1: Generating a list of even numbers

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
res = [num for num in a if num % 2 == 0]
print(res)
```

Output

[2, 4, 6, 8]

Explanation: This creates a list of even numbers by filtering elements from **a** that are divisible by 2.

Example 2: Creating a list of squares

```
res = [num**2 for num in range(1, 6)]
print(res)
```

Output

[1, 4, 9, 16, 25]

Explanation: This generates a list of squares for numbers from 1 to 5.

Dictionary comprehension

Dictionary Comprehensions are used to construct dictionaries in a compact form, making it easy to generate key-value pairs dynamically based on an iterable.

Syntax:

{key_expression: value_expression for item in iterable if condition}

Where:

key_expression: Determines the dictionary key.

value_expression: Computes the value.

iterable: The source collection.

condition (optional): Filters elements before adding them.

Example 1: Creating a dictionary of numbers and their cubes

```
res = {num: num**3 for num in range(1, 6)}
print(res)
```

Output

{1: 1, 2: 8, 3: 27, 4: 64, 5: 125}

Explanation: This creates a dictionary where keys are numbers from 1 to 5 and values are their cubes.

Example 2: Mapping states to capitals

```
a = ["Texas", "California", "Florida"] # states
b = ["Austin", "Sacramento", "Tallahassee"] # capital

res = {state: capital for state, capital in zip(a, b)}
print(res)
```

Output

{'Texas': 'Austin', 'California': 'Sacramento', 'Florida': 'Tallahassee’}

Explanation: zip() function pairs each state with its corresponding capital, creating a dictionary.

Set comprehensions

Set Comprehensions are similar to list comprehensions but result in sets, automatically eliminating duplicate values while maintaining a concise syntax.

Syntax:

{expression for item in iterable if condition}

Where:

expression: The operation applied to each item.

iterable: The source collection.

condition (optional): Filters elements before adding them

Example 1: Extracting unique even numbers

```
a = [1, 2, 2, 3, 4, 4, 5, 6, 6, 7]
res = {num for num in a if num % 2 == 0}
print(res)
```

Output

{2, 4, 6}

Explanation: This creates a set of even numbers from **a**, automatically removing duplicates.

Example 2: Creating a set of squared values

```
res = {num**2 for num in range(1, 6)}
print(res)
```

Output
{1, 4, 9, 16, 25}

Explanation: This generates a set of squares, ensuring each value appears only once.

Parul® University | NAAC GRADE A++

https://paruluniversity.ac.in/